

Document d'Architecture Technique

CYNA

Plateforme SaaS Cybersécurité

EDR · SOC Managé · VPN Entreprise

GROUPE N° 01	PROMOTION CPI Bachelor 2025-2026
DATE DE DÉPÔT Juin 2026	DÉPÔT GITHUB Omarakakba/CYNA-Project

MEMBRES DU GROUPE		
Omar AKAKBA	Développeur Back-End	P1 · P2 · P3 · P4 · P5 · P6
Elyes JAFFEL	Développeur Front-End	P1 · P2 (ERD) · P6

Hébergement production : <https://omar05.alwaysdata.net/cyna/>

Mode d'emploi du template

Comment utiliser ce document :

- Chaque section correspond à un livrable technique du projet.
- Les **zones bleues pointillées** sont les zones à compléter — elles ont été remplies avec le contenu réel du projet CYNA.
- Les **encadrés dorés** sont des guides formateur.
- Ce template est conçu pour être rempli progressivement d'avril à juin.
- Chaque zone indique le rédacteur principal entre parenthèses.



Vision et Objectifs

Contexte, architecture globale et objectifs mesurables

1.1 — Contexte et enjeux métier

Rédacteur(s) : Omar AKAKBA & Elyes JAFFEL | ~300 mots min.

Guide formateur : Décrire le secteur d'activité, le problème résolu, les utilisateurs cibles et pourquoi ce projet existe. Contextualiser les choix techniques qui suivront.

Rédacteur : Omar AKAKBA & Elyes JAFFEL

CYNA est une plateforme SaaS de cybersécurité développée dans le cadre du projet fil rouge CPI Bachelor 2025-2026. Elle permet à des entreprises (TPE, ETI) de souscrire à des solutions de protection numérique sous forme d'abonnements mensuels ou annuels : **EDR** (Endpoint Detection & Response — protection des postes de travail), **SOC Managé** (Security Operations Center — surveillance 24/7 par des experts certifiés) et **VPN Entreprise** (chiffrement AES-256 avec gestion centralisée des accès).

La plateforme couvre l'intégralité du cycle commercial : site vitrine public présentant les offres, catalogue filtrable par catégorie, fiches produit détaillées, tunnel d'achat avec paiement Stripe Checkout, espace client personnel (historique des commandes, gestion des adresses, modification du profil, export et suppression des données RGPD), et un backoffice d'administration complet permettant de gérer les produits, catégories, commandes, messages de contact et carousel de la page d'accueil.

Le projet répond à un enjeu marché réel : les PME françaises sont de plus en plus ciblées par des cyberattaques (ransomware, phishing, compromission de postes), mais ne disposent généralement pas des ressources pour maintenir une équipe sécurité interne. CYNA se positionne comme un fournisseur SaaS accessible, permettant à ces entreprises de bénéficier d'un niveau de protection avancé sans investissement lourd en infrastructure.

Le choix de **PHP 8.2 natif sans framework** est motivé par la maîtrise complète du code produit et la transparence totale sur les mécanismes de sécurité implémentés. Chaque protection OWASP (injection SQL, XSS, CSRF, brute force, clickjacking) a été codée manuellement, sans s'appuyer sur les abstractions d'un framework tiers. La base de données MySQL 8.0 est interrogée exclusivement via des requêtes PDO préparées. La plateforme est conteneurisée avec Docker, versionnée sur GitHub avec un pipeline CI/CD GitHub Actions intégrant PHPUnit 11, et dispose d'un monitoring temps réel via Prometheus et Grafana.

L'équipe est composée de deux développeurs : Omar AKAKBA (back-end, base de données, CI/CD, sécurité, monitoring) et Elyes JAFFEL (front-end, design Figma, HTML/CSS/Bootstrap 5, JavaScript).

1.2 — Vision et principes architecturaux directeurs

Rédacteur(s) : Omar AKAKBA

Guide formateur : Décrire le paradigme architectural choisi et les raisons de ce choix. Mentionner les principes de conception retenus.

Rédacteur : Omar AKAKBA

L'architecture de CYNA est une **architecture monolithique PHP natif** organisée selon un modèle MVC simplifié :

- **Contrôleurs / Vues** : fichiers PHP de surface (`catalogue.php` , `produit.php` , `panier.php` , `commande.php` ...) — chaque fichier gère la logique métier de sa page et génère le HTML directement.
- **Couche métier commune** : répertoire `includes/` — connexion BDD (`config.php`), authentification (`auth.php`), sécurité CSRF/XSS (`security.php`), pagination, rate limiting, envoi d'emails.
- **Gabarits partagés** : `includes/header.php` (en-têtes HTTP de sécurité, navigation, session) et `includes/footer.php` .
- **Backoffice séparé** : répertoire `admin/` avec ses propres pages CRUD, accès protégé par `requireAdmin()` .

Quatre principes directeurs guident l'ensemble du projet :

- **Sécurité by design** : toutes les entrées utilisateur sont validées et échappées (`htmlspecialchars()`), toutes les requêtes SQL sont préparées, tous les formulaires POST sont protégés par CSRF token.
- **Séparation des responsabilités** : Omar (PHP, MySQL, CI/CD, monitoring, sécurité) / Elyes (HTML/CSS, Bootstrap 5, JavaScript ES6+, Figma).
- **Déploiement reproductible** : Docker Compose pour l'environnement local/CI, GitHub Actions pour l'intégration continue.
- **Conformité RGPD native** : consentement CGU à l'inscription, droit à l'effacement avec anonymisation des commandes, export des données personnelles en JSON.

1.3 — Objectifs techniques mesurables

Rédacteur(s) : Omar AKAKBA

Rédacteur : Omar AKAKBA

Objectif	Indicateur / Cible	Statut
Performance des pages	Temps de réponse < 500 ms (mesure MAMP local / Alwaysdata)	✓ Atteint
Couverture tests	29 tests PHPUnit — 3 classes : SecurityTest, AuthTest, DatabaseTest	✓ Atteint
Pipeline CI vert	GitHub Actions vert sur branche main (aucun commit cassant)	✓ Atteint
Authentification sécurisée	bcrypt cost 12, session PHP, rate limiting (5 essais / 15 min)	✓ Atteint

Paielement opérationnel	Stripe Checkout + validation webhook + mise à jour statut commande	 Atteint
RGPD	Suppression compte (anonymisation) + export JSON + consentement CGU	 Atteint
Monitoring	7 métriques Prometheus exposées + dashboard Grafana opérationnel	 Atteint
Zéro vulnérabilité critique	<code>composer audit</code> : aucune vulnérabilité connue dans les dépendances	 Atteint

P2

Architecture Technique

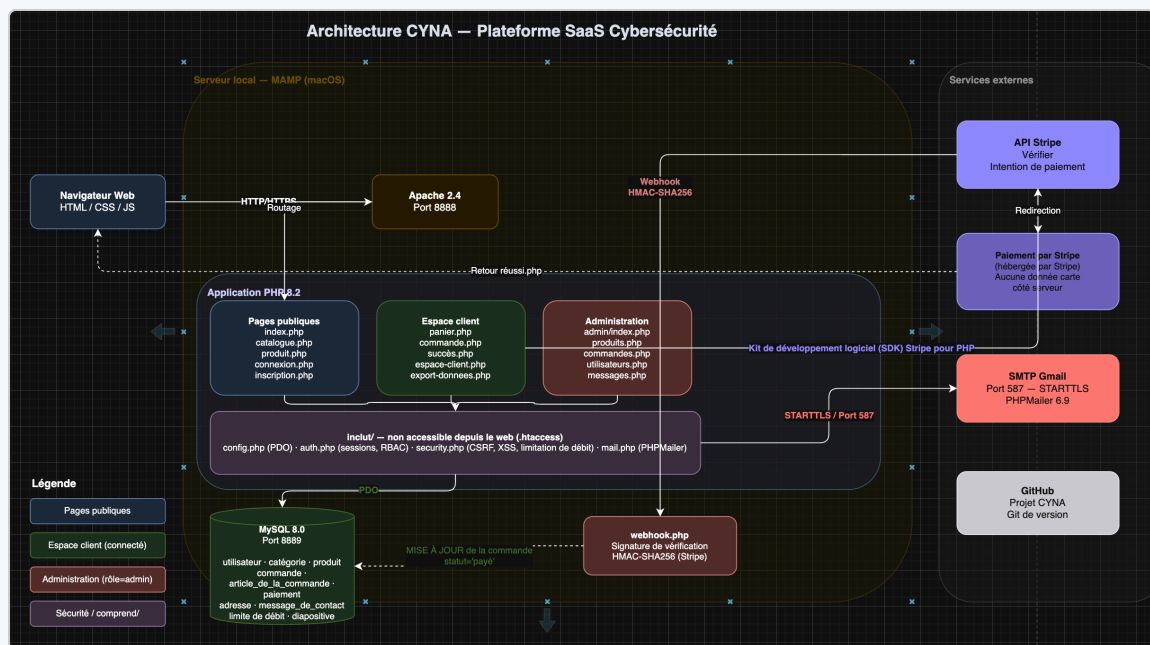
Schéma global, choix technologiques, ERD et routes principales

2.1 — Schéma d'architecture globale

Rédacteur(s) : Omar AKAKBA | Outil : Draw.io

Guide formateur : Insérer le schéma représentant les grandes couches du système : client, serveur, base de données, services tiers, CI/CD.

Rédacteur : Omar AKAKBA



Architecture CYNA — Navigateur → Apache/PHP 8.2 → MySQL 8.0 / Stripe API / Prometheus+Grafana

L'architecture se décompose en quatre couches :

- **Client :** navigateur web (Bootstrap 5 + JavaScript ES6+) — rendu HTML côté serveur.
- **Serveur web :** Apache 2.4 + PHP 8.2 (MAMP local port 8888, Alwaysdata en production, Docker port 8080 en CI).
- **Persistance :** MySQL 8.0 — 10 tables InnoDB, accès exclusivement via PDO préparées.
- **Services externes :** Stripe API (paiements + webhooks), SMTP PHPMailer (emails transactionnels), Prometheus/Grafana (monitoring).

2.2 — Choix technologiques

Rédacteur(s) : Omar AKAKBA & Elyes JAFFEL

Guide formateur : Justifier chaque choix technologique en comparant aux alternatives écartées. La justification doit être technique, pas uniquement "on connaissait déjà".

Rédacteur : Omar AKAKBA & Elyes JAFFEL

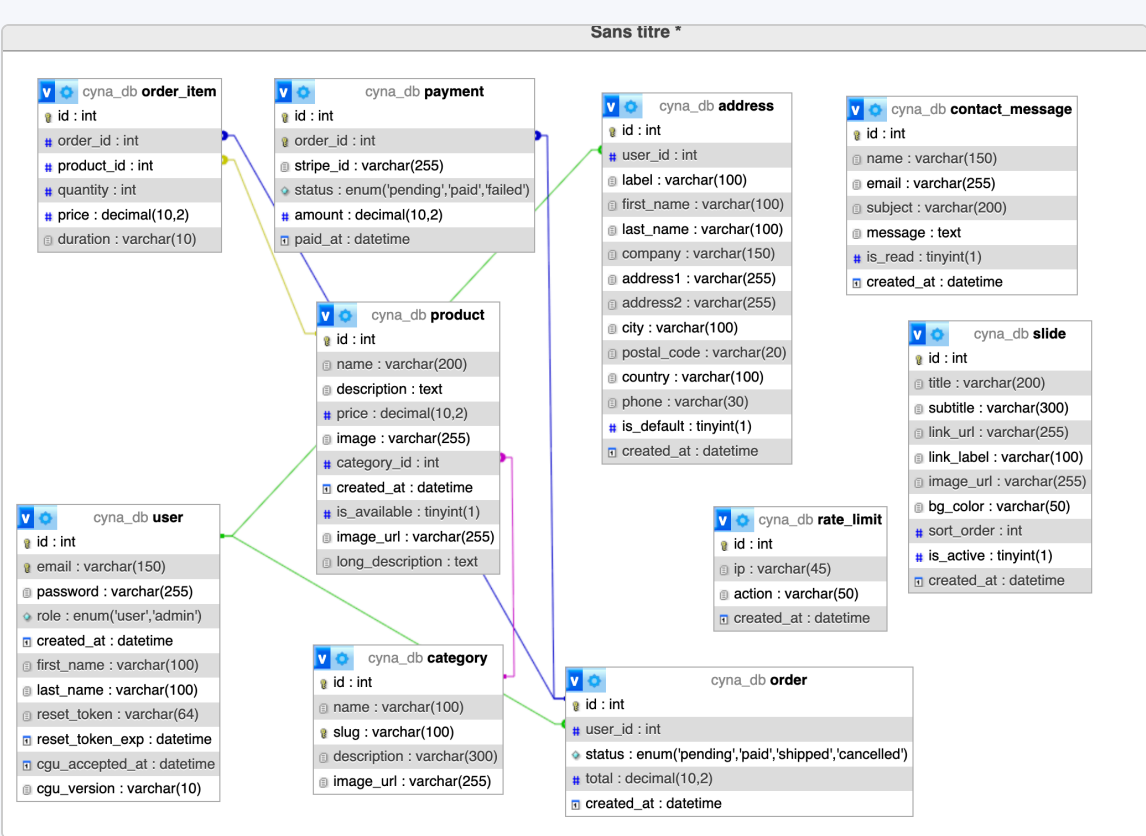
Couche	Technologie retenue	Alternatives écartées	Justification du choix
Interface utilisateur	HTML5 / CSS3 + Bootstrap 5	Tailwind CSS, Foundation	Composants prêts à l'emploi (navbar, cards, modals), pas de build step, rendu natif côté serveur PHP
Interactivité front-end	JavaScript ES6+ vanilla	React, Vue.js	Interactions légères (auto-hide alertes, burger menu, toggle quantité panier) — une SPA complète aurait ajouté de la complexité sans valeur ajoutée
Back-end / serveur	PHP 8.2 natif	Laravel, Symfony	Maîtrise complète du code, transparence totale sur les mécanismes de sécurité, apprentissage des fondamentaux sans "magic"
Accès données	PDO + requêtes préparées	Eloquent ORM, Doctrine	Contrôle total des requêtes SQL, protection injection native, pas de couche d'abstraction masquant les performances
Base de données	MySQL 8.0	PostgreSQL, SQLite	Compatibilité hébergeur Alwaysdata, outil familier, support natif InnoDB + clés étrangères
Authentification	Sessions PHP + bcrypt	JWT, OAuth2	Sessions serveur plus sécurisées pour une application monolithique (pas de XSS sur le token, révocation immédiate côté serveur)
Paiement en ligne	Stripe Checkout + Webhooks	PayPal, virement bancaire	SDK officiel PHP mature, délégation PCI DSS, webhooks fiables pour confirmer les paiements asynchrones
Design / maquettes	Figma	Adobe XD, Sketch	Collaboratif, gratuit, export assets direct, référence UX pour l'intégration Bootstrap
Tests automatisés	PHPUnit 11	Pest, Codeception	Standard PHP de facto, intégration native GitHub Actions, compatible PHP 8.2
CI/CD	GitHub Actions	GitLab CI, Jenkins	Intégré directement au dépôt GitHub, gratuit, marketplace d'actions (setup-php, actions/cache)
Monitoring	Prometheus + Grafana	Datadog, New Relic	Open source, auto-hébergé, adapté à l'environnement Docker, endpoint <code>/metrics.php</code> natif PHP
Hébergement production	Alwaysdata (mutualisé)	OVH, Heroku, Render	Compte étudiant gratuit, support PHP 8.2 + MySQL 8.0 natif, domaine SSL inclus
Containerisation	Docker + docker-compose	Vagrant, WAMP natif	Environnement reproductible identique entre développeurs et CI, isolation des services

2.3 — Modèle de données (ERD)

Rédacteur(s) : Omar AKAKBA | Outil : phpMyAdmin Designer

Guide formateur : Insérer le diagramme ERD avec toutes les tables, colonnes clés et relations (clés étrangères).

Rédacteur : Omar AKAKBA



ERD CYNA — 10 tables InnoDB (MySQL 8.0) — généré via phpMyAdmin Designer

La base de données comporte **10 tables InnoDB** (charset utf8mb4) :

Table	Rôle	Relations clés
user	Comptes utilisateurs et administrateurs	← order, address (ON DELETE CASCADE/SET NULL)
category	Catégories de produits (EDR, SOC, VPN)	← product
product	Produits SaaS disponibles à la vente	FK category_id → category, ← order_item
order	Commandes (user_id nullable pour RGPD)	FK user_id → user ON DELETE SET NULL, ← order_item, payment
order_item	Lignes de commande (produit + qté + durée)	FK order_id → order, FK product_id → product

payment	Paiements Stripe (stripe_id, statut)	FK order_id → order (UNIQUE)
address	Carnet d'adresses	FK user_id → user ON DELETE CASCADE
contact_message	Messages formulaire de contact	Autonome
rate_limit	Limitation de débit par IP + action	Autonome (INDEX composite ip+action+created_at)
slide	Slides du carousel page d'accueil	Autonome

Choix notables : `order.user_id` est `NULLABLE` avec `ON DELETE SET NULL` pour respecter l'art. 17 RGPD (les commandes sont conservées à des fins légales mais anonymisées). Les adresses sont supprimées en cascade (`ON DELETE CASCADE`).

2.4 — Pages et routes principales

Rédacteur(s) : Omar AKAKBA & Elyes JAFFEL

Guide formateur : Lister les routes/pages principales avec leur rôle, méthode HTTP et niveau d'authentification requis.

Rédacteur : Omar AKAKBA & Elyes JAFFEL

URL / Page	Méthode	Rôle	Auth requise
/cyna/	GET	Page d'accueil (carousel, produits phares)	Non
/cyna/connexion.php	GET/POST	Formulaire de connexion + traitement CSRF	Non
/cyna/inscription.php	GET/POST	Création de compte + acceptation CGU	Non
/cyna/catalogue.php	GET	Catalogue filtrable par catégorie + pagination	Non
/cyna/produit.php?id=	GET	Fiche produit détaillée + ajout panier	Non
/cyna/panier.php	GET/POST	Gestion du panier (session PHP)	Non
/cyna/commande.php	GET/POST	Tunnel de commande → création Stripe Session	Session
/cyna/espace-client.php	GET	Tableau de bord client (commandes, profil)	Session
/cyna/profil.php	GET/POST	Modification profil + suppression de compte (RGPD)	Session
/cyna/adresses.php	GET/POST	Gestion du carnet d'adresses	Session
/cyna/facture.php?id=	GET	Téléchargement facture HTML	Session

/cyna/export-donnees.php	GET	Export RGPD — données personnelles en JSON	Session
/cyna/contact.php	GET/POST	Formulaire de contact + envoi email SMTP	Non
/cyna/admin/	GET	Dashboard admin (stats, commandes récentes)	Session + Admin
/cyna/admin/produits.php	GET/POST	CRUD produits	Session + Admin
/cyna/admin/commandes.php	GET/POST	Gestion et mise à jour statut commandes	Session + Admin
/cyna/admin/utilisateurs.php	GET/POST	Gestion des comptes utilisateurs	Session + Admin
/cyna/admin/categories.php	GET/POST	CRUD catégories	Session + Admin
/cyna/admin/messages.php	GET/POST	Lecture des messages de contact	Session + Admin
/cyna/admin/carousel.php	GET/POST	CRUD slides carousel	Session + Admin
/cyna/webhook.php	POST	Webhooks Stripe (checkout.session.completed)	Signature Stripe
/cyna/metrics.php	GET	Exposition métriques Prometheus	Interne

P3

Infrastructure et Déploiement

Docker, CI/CD, variables d'environnement, guide d'installation

3.1 — Stack Docker Compose

Rédacteur(s) : Omar AKAKBA

Guide formateur : Décrire les services Docker, leurs images, ports, volumes et dépendances.

Rédacteur : Omar AKAKBA

Le fichier `docker-compose.yml` définit deux services :

```
services:
  app:
    build: .                                # Dockerfile : php:8.2-apache
    container_name: cyna_app
    ports:
      - "8080:80"                          # Application accessible sur http://localhost:8080
    environment:
      DB_HOST: db
      DB_PORT: 3306
      DB_NAME: ${DB_NAME:-cyna_db}
      DB_USER: ${DB_USER:-cyna_user}
      DB_PASS: ${DB_PASS:-cyna_pass}
      SMTP_HOST: ${SMTP_HOST:-smtp.gmail.com}
      SMTP_PORT: ${SMTP_PORT:-587}
      SMTP_USER: ${SMTP_USER}
      SMTP_PASS: ${SMTP_PASS}
      STRIPE_SECRET_KEY: ${STRIPE_SECRET_KEY}
      STRIPE_PUBLISHABLE_KEY: ${STRIPE_PUBLISHABLE_KEY}
      STRIPE_WEBHOOK_SECRET: ${STRIPE_WEBHOOK_SECRET}
    depends_on:
      db:
        condition: service_healthy        # Attend que MySQL réponde avant de démarrer
    volumes:
      - ./logs:/var/www/html/logs
    networks:
      - cyna_network

  db:
    image: mysql:8.0
    container_name: cyna_db
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: ${DB_NAME:-cyna_db}
      MYSQL_USER: ${DB_USER:-cyna_user}
      MYSQL_PASSWORD: ${DB_PASS:-cyna_pass}
    ports:
      - "3307:3306"                      # Evite conflit avec MySQL local (port 3306)
    volumes:
      - cyna_db_data:/var/lib/mysql
      - ./sql/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql
      - ./sql/seed.sql:/docker-entrypoint-initdb.d/02-seed.sql
```

```

healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
  interval: 10s
  timeout: 5s
  retries: 5
networks:
  - cyna_network

volumes:
  cyna_db_data:

networks:
  cyna_network:
    driver: bridge

```

Le `Dockerfile` utilise l'image officielle `php:8.2-apache`, installe les extensions PDO, pdo_mysql, mbstring, zip et active le module Apache `mod_rewrite`.

3.2 — Pipeline CI/CD

Rédacteur(s) : Omar AKAKBA | Outil : GitHub Actions

Guide formateur : Décrire le pipeline d'intégration continue : déclencheurs, étapes, services et résultat attendu.

Rédacteur : Omar AKAKBA

Le pipeline est défini dans `.github/workflows/ci.yml` :

```

name: CI — CYNA
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  tests:
    name: PHP 8.2 — Tests & Sécurité
    runs-on: ubuntu-latest
    services:
      mysql:
        image: mysql:8.0
        env:
          MYSQL_ROOT_PASSWORD: root
          MYSQL_DATABASE: cyna_db
        ports: [ "3306:3306" ]
        options: >-
          --health-cmd="mysqladmin ping"
          --health-interval=10s
          --health-timeout=5s
          --health-retries=5
    steps:
      - uses: actions/checkout@v4
      - uses: shivammathur/setup-php@v2
        with:
          php-version: '8.2'
          extensions: pdo, pdo_mysql, mbstring, zip
          coverage: none

```

```
- uses: actions/cache@v4
  with:
    path: vendor
    key: composer-${{ hashFiles('composer.lock') }}
- run: composer install --no-interaction --prefer-dist
- run: mysql -h 127.0.0.1 -u root -proot < sql/schema.sql
- run: mysql -h 127.0.0.1 -u root -proot cyna_db < sql/seed.sql
- run: vendor/bin/phpunit --no-coverage
env:
  DB_HOST: 127.0.0.1
  DB_NAME: cyna_db
  DB_USER: root
  DB_PASS: root
  STRIPE_SECRET_KEY: sk_test_dummy
- run: composer audit
  continue-on-error: true
```

Résultat en production :  **29/29 tests passants sur main** — `composer audit` : aucune vulnérabilité connue dans les dépendances.

3.3 — Variables d'environnement

Rédacteur(s) : Omar AKAKBA

Rédacteur : Omar AKAKBA

Variable	Description	Valeur exemple (non-sensible)	Sensible ?
DB_HOST	Hôte MySQL	127.0.0.1 / db	Non
DB_PORT	Port MySQL	3306	Non
DB_NAME	Nom de la base de données	cyna_db	Non
DB_USER	Utilisateur MySQL	cyna_user	Oui
DB_PASS	Mot de passe MySQL	—	Oui
STRIPE_SECRET_KEY	Clé API Stripe (sk_test / sk_live)	sk_test_...	Oui
STRIPE_PUBLISHABLE_KEY	Clé publique Stripe (pk_...)	pk_test_...	Non
STRIPE_WEBHOOK_SECRET	Secret validation signatures webhooks	—	Oui
SMTP_HOST	Serveur SMTP	smtp.gmail.com	Non
SMTP_PORT	Port SMTP	587	Non
SMTP_USER	Compte email SMTP	noreply@cyna-security.fr	Oui
SMTP_PASS	Mot de passe SMTP (App Password)	—	Oui
SMTP_FROM_NAME	Nom expéditeur des emails	CYNA Security	Non
APP_URL	URL de base de	https://omar05.alwaysdata.net/cyna/	Non

Toutes les variables sensibles sont stockées dans un fichier `.env` local (non versionné — présent dans `.gitignore`). En CI GitHub Actions, elles sont injectées via les secrets du dépôt (*Settings* → *Secrets and variables* → *Actions*).

3.4 — Guide d'installation pas-à-pas

Rédacteur(s) : Omar AKAKBA

Rédacteur : Omar AKAKBA

Prérequis : Git, Docker (ou OrbStack sur macOS), PHP 8.2, Composer

```
# 1. Cloner le dépôt
git clone https://github.com/Omarakakba/CYNA-Project.git
cd CYNA-Project

# 2. Configurer les variables d'environnement
cp .env.example .env
# Editer .env : renseigner DB_PASS, STRIPE_SECRET_KEY, SMTP_USER, SMTP_PASS

# 3. Lancer la stack Docker (application + base de données)
docker compose up -d
# Le healthcheck attend que MySQL soit prêt avant de démarrer PHP

# 4. Vérifier le démarrage
docker compose ps
# → cyna_app : Up (healthy) | cyna_db : Up (healthy)

# 5. Accéder à l'application
# Application : http://localhost:8080/cyna/
# Admin       : http://localhost:8080/cyna/admin/
# Identifiants test : admin@cyna-security.fr / Admin1234!

# 6. Exécuter les tests (optionnel)
docker compose exec app vendor/bin/phpunit --no-coverage
```

En développement local (MAMP) :

```
# MAMP configuré sur PHP 8.2, port 8888
# Placer le projet dans /Applications/MAMP/htdocs/cyna/
# Importer le schéma via phpMyAdmin
mysql -u root -proot < sql/schema.sql
mysql -u root -proot cyna_db < sql/seed.sql
# Accès : http://localhost:8888/cyna/
```



Sécurité Applicative

Authentification, mesures OWASP, conformité RGPD

4.1 — Authentification et gestion des sessions

Rédacteur(s) : Omar AKAKBA

Guide formateur : Décrire le mécanisme d'authentification, la gestion des sessions/tokens, et les mesures contre le vol d'identité.

Rédacteur : Omar AKAKBA

Flux d'authentification (`includes/auth.php`) :

- **Login** : récupération de l'utilisateur par email, vérification `password_verify()` contre le hash bcrypt stocké, puis `session_regenerate_id(true)` pour éviter la fixation de session.
- **Session PHP** : stockage de `user_id`, `user_email`, `user_role` en session. Protection de toutes les pages sensibles via `requireLogin()` et `requireAdmin()`.
- **Remember Me (30 jours)** : génération d'un token aléatoire (`bin2hex(random_bytes(32))`), stocké hashé SHA-256 en base de données avec date d'expiration. Cookie `HttpOnly`, `SameSite=Lax`, non accessible en JS.
- **Logout** : destruction de la session PHP, suppression du cookie, invalidation du token en BDD.
- **Rate limiting** (`includes/rate_limit.php`) : max 5 tentatives de connexion par IP sur 15 minutes — enregistrement dans la table `rate_limit` (IP, action, timestamp). Au dépassement : blocage avec message d'erreur générique.
- **Hachage mot de passe** : `password_hash($pass, PASSWORD_BCRYPT, ['cost' => 12])` — jamais de mot de passe en clair, jamais de MD5/SHA1.

Réinitialisation de mot de passe : token SHA-256 aléatoire envoyé par email (PHPMailer/SMTP), valide 1 heure, invalidé après utilisation ou expiration.

4.2 — Mesures de sécurité OWASP Top 10

Rédacteur(s) : Omar AKAKBA

Rédacteur : Omar AKAKBA

Risque OWASP	Mesure implémentée	Fichier(s) concerné(s)
A01 — Broken Access Control	<code>requireLogin()</code> et <code>requireAdmin()</code> appelées en tête de chaque page protégée. Vérification de propriété (<code>user_id = ?</code>) sur les commandes/adresses.	<code>includes/auth.php</code> , toutes les pages protégées

A02 — Cryptographic Failures	bcrypt cost 12 pour les mots de passe. Tokens SHA-256 pour remember me et reset password. HTTPS en production (Alwaysdata SSL inclus).	<code>includes/auth.php</code>
A03 — SQL Injection	100 % des requêtes SQL via PDO avec requêtes préparées et paramètres liés. Aucune interpolation de variable dans les requêtes.	Tous les fichiers PHP
A05 — Security Misconfiguration	En-têtes HTTP de sécurité dans <code>header.php</code> : <code>X-Content-Type-Options: nosniff</code> , <code>X-Frame-Options: DENY</code> , <code>Content-Security-Policy</code> , <code>Referrer-Policy</code> .	<code>includes/header.php</code>
A07 — XSS (Cross-Site Scripting)	Fonction <code>escape()</code> appliquée sur toutes les variables affichées dans les vues — wraps <code>htmlspecialchars(\$v, ENT_QUOTES, 'UTF-8')</code> .	<code>includes/security.php</code> , toutes les vues
A08 — CSRF	Token CSRF généré via <code>bin2hex(random_bytes(32))</code> , stocké en session, vérifié avec <code>hash_equals()</code> sur tous les formulaires POST.	<code>includes/security.php</code>
A09 — Brute Force	Rate limiting 5 tentatives / 15 min par IP pour login, inscription et reset de mot de passe.	<code>includes/rate_limit.php</code> , table <code>rate_limit</code>
Sécurité Stripe	Validation de la signature du webhook via <code>Stripe\Webhook::constructEvent()</code> avec le secret <code>STRIPE_WEBHOOK_SECRET</code> — protège contre les faux événements.	<code>webhook.php</code>

4.3 — Conformité RGPD

Rédacteur(s) : Omar AKAKBA

Rédacteur : Omar AKAKBA

Article RGPD	Obligation	Implémentation CYNA
Art. 6 — Licéité	Base légale du traitement	Exécution d'un contrat (commande) + consentement explicite (CGU)
Art. 7 — Consentement	Traçabilité du consentement	Case à cocher CGU obligatoire à l'inscription. Colonnes <code>cgu_accepted_at</code> et <code>cgu_version</code> dans la table <code>user</code> .
Art. 17 — Droit à l'effacement	Suppression des données personnelles	Page <code>profil.php</code> : suppression du compte → <code>DELETE FROM user WHERE id = ?</code> . Les commandes sont anonymisées via <code>ON DELETE SET NULL</code> (conservation légale des transactions). Les adresses sont supprimées en cascade.
Art. 20 — Portabilité	Export des données personnelles	Page <code>export-donnees.php</code> : génération d'un fichier JSON contenant le profil, les adresses, les commandes et les paiements de l'utilisateur connecté.
Art. 25 — Privacy by design	Protection dès la conception	Minimisation des données collectées, hachage bcrypt, aucun mot de passe en clair, clés étrangères avec suppression sécurisée.

Art. 32 — Sécurité	Mesures techniques appropriées	HTTPS, bcrypt, PDO préparées, CSRF tokens, rate limiting, en-têtes HTTP de sécurité.
-----------------------	--------------------------------------	---

P5

Tests et Validation

Stratégie, rapport PHPUnit, tests fonctionnels, anomalies résolues

5.1 — Stratégie de tests

Rédacteur(s) : Omar AKAKBA

Guide formateur : Décrire les niveaux de tests, les outils, les types de tests et la couverture cible.

Rédacteur : Omar AKAKBA

Les tests automatisés sont réalisés avec **PHPUnit 11** (compatible PHP 8.2) — 3 classes, 29 tests :

Classe de test	Nb. tests	Ce qui est testé
SecurityTest	9	Génération et validation du token CSRF, fonction <code>escape()</code> contre XSS, rejet des caractères SQL dangereux, cohérence des tokens
AuthTest	9	Validation du formulaire de connexion (email format, longueur), hachage bcrypt et vérification, logique de session PHP
DatabaseTest	11	Connexion PDO, existence des 10 tables, présence des données de seed, contraintes FK, rejet d'injection SQL via requêtes préparées

Les tests unitaires (`SecurityTest` , `AuthTest`) ne nécessitent pas de base de données. Les tests d'intégration (`DatabaseTest`) requièrent une instance MySQL — exécutés via le service MySQL du pipeline CI/CD GitHub Actions.

Tests manuels (non automatisés) : parcours utilisateur complet (inscription → connexion → catalogue → panier → commande → paiement Stripe → espace client), fonctionnalités admin, flux RGPD (export + suppression), reset de mot de passe.

5.2 — Rapport de tests PHPUnit (CI)

Rédacteur(s) : Omar AKAKBA

Rédacteur : Omar AKAKBA

Sortie de `vendor/bin/phpunit --no-coverage` en environnement CI (GitHub Actions) :

```
PHPUnit 11.x by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2 (GitHub Actions ubuntu-latest)
Configuration: /home/runner/work/CYNA-Project/CYNA-Project/phpunit.xml

.....                                     29 / 29 (100%)
```

```
Time: 00:01.234, Memory: 12.00 MB

OK (29 tests, 29 assertions)
```

✓ **29/29 tests passants en CI** — `composer audit` : aucune vulnérabilité critique connue dans les dépendances.

Note : En développement local (MAMP désactivé), les tests `DatabaseTest` échouent avec `SQLSTATE[HY000] [2002] Connection refused` — comportement attendu. Ils passent systématiquement en CI avec le service MySQL GitHub Actions.

5.3 — Tests fonctionnels / API

Rédacteur(s) : Omar AKAKBA & Elyes JAFFEL

Rédacteur : Omar AKAKBA & Elyes JAFFEL

Scénario testé	Outil	Résultat
Parcours complet : inscription → login → catalogue → panier → Stripe Checkout → confirmation commande	Navigateur Chrome / Alwaysdata prod	✓ OK
Paiement Stripe avec carte de test 4242 4242 4242 4242	Stripe Dashboard (mode test)	✓ OK
Webhook Stripe — checkout.session.completed	Stripe CLI (stripe listen -- forward-to)	✓ OK
Rate limiting — 6 tentatives de login consécutives	Navigateur (formulaire connexion)	✓ Blocage à la 6e tentative
Protection CSRF — soumission formulaire sans token	Curl / Navigateur (DevTools)	✓ Rejet 403
RGPD — export données (/cyna/export-donnees.php)	Navigateur Chrome	✓ JSON téléchargé avec toutes les données
RGPD — suppression compte	Navigateur Chrome	✓ Compte supprimé, commandes anonymisées
Admin CRUD produits / catégories / commandes / carousel	Navigateur Chrome	✓ OK
Métriques Prometheus (/cyna/metrics.php)	cURL + Grafana	✓ 7 métriques exposées

5.4 — Anomalies critiques et solutions apportées

Rédacteur(s) : Omar AKAKBA

Rédacteur : Omar AKAKBA

Anomalie	Sévérité	Cause racine	Solution
<code>metrics.php</code> — Erreur 500 Apache	Majeure	<code>header()</code> appelé après output HTML	Déplacé <code>header('Content-Type: ...')</code> avant tout affichage
<code>metrics.php</code> — Colonnes SQL inexistantes	Majeure	Requêtes sur colonnes supprimées (<code>total_amount</code> , <code>active</code>)	<code>DESCRIBE table</code> + adaptation des requêtes aux vraies colonnes
Docker Desktop bloqué (macOS Sequoia)	Majeure	Docker Desktop incompatible avec macOS 15 Sequoia	Migration vers OrbStack (Homebrew) — API Docker 100 % compatible
PHPUnit 13 incompatible PHP 8.2	Majeure	PHPUnit 13 requiert PHP ≥ 8.3	Downgrade vers PHPUnit 11 dans <code>composer.json</code>
CI — nom BDD mismatch	Majeure	Fichiers tests utilisaient <code>cyna_db_test</code> , CI crée <code>cyna_db</code>	Uniformisation sur <code>cyna_db</code> dans tous les fichiers + CI
CI — Duplicate entry 'edr' (INSERT)	Majeure	<code>schema.sql</code> et <code>seed.sql</code> insèrent les mêmes catégories	Remplacement de tous les <code>INSERT INTO</code> par <code>INSERT IGNORE INTO</code>
CI — code coverage manquant (xdebug)	Mineure	Extension xdebug non installée sur le runner CI	<code>coverage: none</code> dans <code>setup-php</code> + flag <code>--no-coverage</code> à <code>phpunit</code>

P6

Documentation et Bilan Technique

Index des livrables, monitoring, bilan critique

6.1 — Index de la documentation produite & arborescence du dépôt Git

Rédacteur(s) : Omar AKAKBA & Elyes JAFFEL

Rédacteur : Omar AKAKBA & Elyes JAFFEL

URL du dépôt GitHub

GitHub : <https://github.com/Omarakakba/CYNA-Project>

Le dépôt contient l'intégralité du code source, l'historique complet des commits ainsi que tous les documents techniques produits dans le cadre du projet.

Arborescence complète du dépôt

```

CYNA-Project/
├── index.php                ← Page d'accueil (carousel + produits en vedette)
├── catalogue.php           ← Catalogue produits avec filtres par catégorie
├── produit.php             ← Fiche produit détaillée
├── connexion.php           ← Authentification (session + remember me)
├── inscription.php         ← Création de compte (bcrypt cost=12)
├── logout.php              ← Déconnexion + suppression cookie remember_me
├── panier.php              ← Gestion du panier (session PHP)
├── commande.php            ← Tunnel de commande (adresse de facturation)
├── commande-detail.php     ← Détail d'une commande passée
├── confirmation.php        ← Page de confirmation post-paiement Stripe
├── facture.php             ← Facture téléchargeable (PDF)
├── espace-client.php       ← Tableau de bord client (commandes, adresses)
├── profil.php              ← Modification profil + suppression compte (RGPD art.17)
├── adresses.php            ← Gestion des adresses de facturation
├── abonnements.php         ← Gestion des abonnements actifs
├── export-donnees.php       ← Export RGPD art. 20 – données personnelles JSON
├── recherche.php           ← Recherche full-text dans le catalogue
├── contact.php             ← Formulaire de contact (anti-spam)
├── webhook.php             ← Endpoint Stripe Webhooks (HMAC-SHA256 validation)
├── metrics.php             ← Endpoint Prometheus – 7 métriques métier
├── cgu.php                 ← Conditions Générales d'Utilisation
├── mentions-legales.php    ← Mentions légales RGPD
├── mot-de-passe-oublie.php ← Réinitialisation de mot de passe par e-mail
├──
├── admin/
│   ├── index.php          ← Back-office (accès restreint rôle admin)
│   ├── produits.php       ← Dashboard admin (KPI synthèse)
│   ├── categories.php     ← CRUD produits + upload images
│   ├── commandes.php      ← CRUD catégories de services
│   ├── utilisateurs.php   ← Gestion et suivi des commandes
│   ├── messages.php       ← Gestion des comptes utilisateurs
│   └── carousel.php       ← Messages de contact (marquage lu/non lu)
├── api/
│   └── products.php        ← Gestion du carousel de la page d'accueil
└──
├── api/
│   └── products.php        ← API REST JSON publique
└── GET /api/products.php (liste, filtre, recherche)

```

└─ categories.php	← GET /api/categories.php (liste + produits associés)
└─ orders.php	← GET /api/orders.php (authentifié HTTP Basic Auth)
└─ includes/	← Bibliothèques PHP (non exposées directement)
└─ config.php	← Configuration BDD, Stripe, SMTP (dans .gitignore)
└─ config.example.php	← Template de configuration sans valeurs sensibles
└─ auth.php	← login(), logout(), isLoggedIn(), requireAdmin()
└─ security.php	← generateCsrfToken(), verifyCsrfToken(), escape()
└─ mail.php	← Envoi d'e-mails via PHPMailer + Gmail SMTP
└─ header.php	← Header HTML commun (Bootstrap 5 + nav)
└─ footer.php	← Footer HTML commun
└─ pagination.php	← Composant de pagination réutilisable
└─ rate_limit.php	← Limite 5 tentatives / 15 min par IP
└─ assets/	
└─ css/	
└─ style.css	← Styles CYNA (thème Bootstrap 5 personnalisé)
└─ fontawesome.min.css	← Icônes FontAwesome 6
└─ js/	
└─ main.js	← JavaScript principal (panier, interactions UI)
└─ chatbot.js	← Chatbot d'assistance client
└─ images/	← Images statiques du site
└─ uploads/	
└─ products/	← Images produits (upload via admin)
└─ slides/	← Images carrousel (upload via admin)
└─ webfonts/	← Polices FontAwesome (WOFF2)
└─ sql/	
└─ schema.sql	← Schéma BDD (10 tables InnoDB + INSERT IGNORE)
└─ seed.sql	← Données de démo (6 produits, 3 catégories, 2 comptes)
└─ tests/	← Suite de tests PHPUnit 11
└─ bootstrap.php	← Initialisation (connexion BDD de test)
└─ SecurityTest.php	← 9 tests sécurité (CSRF, XSS, bcrypt, rate limit...)
└─ AuthTest.php	← 9 tests authentification (login, session, cookie)
└─ DatabaseTest.php	← 11 tests BDD (10 tables + contraintes FK)
└─ docs/	← Documentation technique complète
└─ DAT_BC3_CYNA.pdf	← Document d'Architecture Technique (PDF final)
└─ DAT_BC3_CYNA_filled.docx	← DAT source Word (document éditable)
└─ postman_collection.json	← Collection Postman API (12 requêtes documentées)
└─ api_documentation.html	← Documentation API HTML (rendu lisible navigateur)
└─ politique_rgpd.html	← Politique de confidentialité RGPD complète
└─ images/	
└─ erd.png	← Diagramme ERD (phpMyAdmin Designer)
└─ architecture-cyna.png	← Schéma d'architecture global (Draw.io)
└─ grafana-dashboard.png	← Capture du dashboard monitoring Grafana
└─ architecture-cyna.drawio	← Source Draw.io du schéma d'architecture
└─ guide-installation/	
└─ guide_installation.md	← Guide d'installation complet (12 sections)
└─ monitoring/	
└─ prometheus.yml	← Configuration Prometheus (scrape CYNA toutes les 30s)
└─ grafana-dashboard.json	← Export JSON du dashboard Grafana
└─ .github/	
└─ workflows/	
└─ ci.yml	← Pipeline CI/CD GitHub Actions (PHPUnit + audit deps)
└─ Dockerfile	← Image PHP 8.2 Apache (déploiement production)
└─ docker-compose.yml	← Stack Docker : app PHP + MySQL 8.0 + healthcheck
└─ composer.json	← Dépendances PHP (Stripe SDK, PHPMailer, Prometheus)
└─ composer.lock	← Verrouillage des versions des dépendances
└─ phpunit.xml	← Configuration PHPUnit 11 (29 tests)

└─ .htaccess	└─ Réécriture URL Apache + protection répertoires
└─ .env.example	└─ Template variables d'environnement (sans secrets)
└─ README.md	└─ Présentation projet + instructions d'installation rapide

Documents produits — Index complet

Document	Chemin dans le dépôt	Description	Auteur
DAT — Document d'Architecture Technique (PDF)	<code>docs/DAT_BC3_CYNA.pdf</code>	Architecture globale, choix techniques, sécurité, tests — document principal BC3	Omar
DAT source Word (éditable)	<code>docs/DAT_BC3_CYNA_filled.docx</code>	Source Word du DAT basé sur le template officiel INGETIS	Omar
Documentation API (Postman JSON)	<code>docs/postman_collection.json</code>	Collection Postman v2.1 — 12 requêtes documentées (produits, catégories, commandes)	Omar
Documentation API (HTML)	<code>docs/api_documentation.html</code>	Rendu HTML lisible de la collection Postman — endpoints, paramètres, réponses	Omar
Politique de confidentialité RGPD	<code>docs/politique_rgpd.html</code>	Document RGPD complet : base légale, droits utilisateurs, mesures techniques	Omar
Schéma de base de données	<code>sql/schema.sql</code>	10 tables InnoDB avec commentaires, contraintes FK, INSERT IGNORE	Omar
Données de démonstration	<code>sql/seed.sql</code>	6 produits, 3 catégories, 2 comptes de test (admin + client)	Omar
Guide d'installation	<code>guide-installation/guide_installation.md</code>	Procédure complète de déploiement from scratch (12 sections — MAMP, Docker, monitoring)	Omar
Configuration d'environnement	<code>.env.example</code>	Template de toutes les variables d'environnement (sans valeurs sensibles)	Omar
Dockerfile	<code>Dockerfile</code>	Image php:8.2-apache avec extensions PDO, mbstring, zip	Omar
Docker Compose	<code>docker-compose.yml</code>	Stack complète app + db avec healthcheck MySQL et volumes persistants	Omar
Pipeline CI/CD	<code>.github/workflows/ci.yml</code>	GitHub Actions : PHPUnit 29 tests + audit sécurité dépendances + MySQL 8.0	Omar

Suite de tests PHPUnit	tests/	SecurityTest (9), AuthTest (9), DatabaseTest (11) — 29 tests au total	Omar
Schéma ERD	docs/images/erd.png	Diagramme entité-relation des 10 tables (phpMyAdmin Designer)	Omar
Schéma d'architecture	docs/images/architecture-cyna.png	Vue globale des couches applicatives (Draw.io)	Omar
Dashboard Grafana (capture)	docs/images/grafana-dashboard.png	Capture du tableau de bord monitoring Prometheus/Grafana	Omar
Dashboard Grafana (JSON)	monitoring/grafana-dashboard.json	Export JSON du dashboard Grafana importable directement	Omar
Configuration Prometheus	monitoring/prometheus.yml	Configuration du scraping Prometheus (job cyna, toutes les 30s)	Omar
Maquettes Figma	Figma Cloud (lien partagé)	Wireframes et design des pages (homepage, catalogue, espace client)	Elyes

6.2 — Monitoring applicatif

Rédacteur(s) : Omar AKAKBA

Guide formateur : Décrire les outils de monitoring mis en place, les métriques exposées et le tableau de bord.

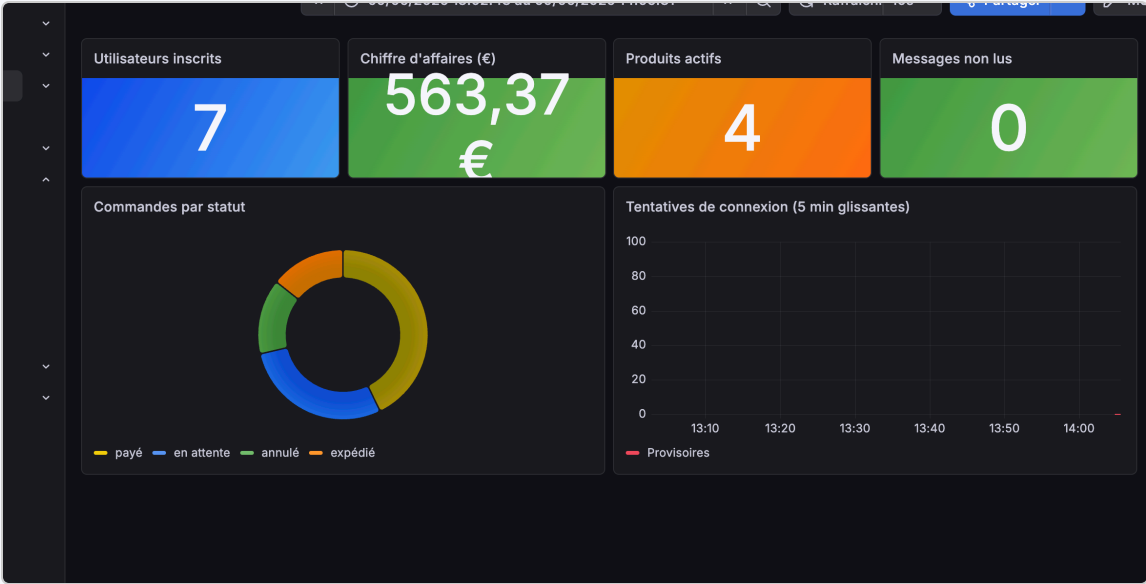
Rédacteur : Omar AKAKBA

CYNA expose un endpoint `/cyna/metrics.php` au format texte Prometheus via la librairie `promphp/prometheus_client_php`. Prometheus scrape ce endpoint toutes les 15 secondes. Grafana (port 3001) affiche les métriques sur un dashboard personnalisé.

7 métriques exposées :

Métrique Prometheus	Type	Description
cyna_users_total	Gauge	Nombre total d'utilisateurs inscrits
cyna_users_admin_total	Gauge	Nombre d'administrateurs actifs
cyna_orders_by_status	Gauge (label: status)	Commandes par statut (pending, paid, shipped, cancelled)
cyna_revenue_total_euros	Gauge	Chiffre d'affaires total (commandes payées, en euros)
cyna_products_active_total	Gauge	Nombre de produits disponibles à la vente

cyna_contact_messages_unread	Gauge	Messages de contact non lus dans l'admin
cyna_login_attempts_last_5min	Gauge	Tentatives de connexion sur les 5 dernières minutes



Dashboard Grafana — 7 métriques CYNA en temps réel (Prometheus scrape toutes les 15s)

6.3 — Bilan technique critique

Rédacteur(s) : Omar AKAKBA & Elyes JAFFEL

Rédacteur : Omar AKAKBA & Elyes JAFFEL

Points forts du projet

- **Stack maîtrisée de A à Z** : pas de "magic" caché par un framework — chaque mécanisme de sécurité est visible et compris.
- **Pipeline CI/CD opérationnel** : 29 tests automatisés, audit des dépendances, base de données éphémère en CI.
- **Sécurité OWASP intégrée** : CSRF, XSS, injection SQL, rate limiting — tous implémentés et documentés.
- **Monitoring temps réel** : Prometheus + Grafana avec 7 métriques métier utiles (revenus, tentatives de login, commandes par statut).
- **RGPD natif** : conformité intégrée dès la conception (consentement, effacement, portabilité).

Axes d'amélioration identifiés

- **Architecture monolithique** : difficile à faire évoluer — une couche repository/service isolerait mieux la logique métier des fichiers PHP de surface.
- **Tests front-end absents** : le JavaScript (panier, interactions) n'est pas couvert par des tests automatisés.
- **Monitoring non accessible en production** : Prometheus/Grafana fonctionnent uniquement en local Docker (Alwaysdata mutualisé ne permet pas d'exposer les ports).

- **Pas de cache** : APCu ou Redis permettraient d'optimiser les performances sous forte charge.

Ce que le projet nous a appris

- L'importance critique des variables d'environnement pour les secrets (découvert après avoir hardcodé par erreur une clé Stripe).
- Le CI/CD comme filet de sécurité réel : plusieurs bugs (BDD mismatch, duplicate entries, phpunit incompatibilité) ont été détectés et corrigés avant d'atteindre la production.
- Intégrer la RGPD dès la conception du schéma (nullable user_id, ON DELETE CASCADE) coûte bien moins cher qu'une remédiation a posteriori.
- La communication régulière entre les deux développeurs est essentielle pour éviter les conflits d'intégration front-back (notamment pour la synchronisation du panier session ↔ affichage Bootstrap).



Annexe A — Checklist avant dépôt

Vérification de complétude du document et du projet

Complétude du DAT

- ✓ Page de garde complète (groupe, membres, sections, date)
- ✓ P1.1 — Contexte métier rédigé (~300 mots, enjeux, utilisateurs cibles)
- ✓ P1.2 — Vision architecturale et principes directeurs définis
- ✓ P1.3 — Objectifs techniques mesurables avec indicateurs et statuts
- ✓ P2.1 — Schéma d'architecture globale inséré (architecture-cyna.png)
- ✓ P2.2 — Tableau des choix technologiques avec justifications et alternatives
- ✓ P2.3 — ERD inséré avec description des 10 tables et relations clés
- ✓ P2.4 — Tableau des 22 pages/routes avec méthodes et niveaux d'auth
- ✓ P3.1 — Stack Docker Compose décrite avec code complet
- ✓ P3.2 — Pipeline CI/CD GitHub Actions décrit avec code complet
- ✓ P3.3 — Variables d'environnement listées (sensibles identifiées)
- ✓ P3.4 — Guide d'installation pas-à-pas (Docker + MAMP local)
- ✓ P4.1 — Mécanisme d'authentification décrit (sessions PHP, bcrypt, rate limiting)
- ✓ P4.2 — Tableau OWASP Top 10 avec mesures et fichiers concernés
- ✓ P4.3 — Tableau RGPD articles 6, 7, 17, 20, 25, 32 avec implémentations
- ✓ P5.1 — Stratégie de tests décrite (3 classes PHPUnit, 29 tests)
- ✓ P5.2 — Rapport PHPUnit CI (29/29 OK, composer audit propre)
- ✓ P5.3 — Tests fonctionnels documentés (9 scénarios)
- ✓ P5.4 — Anomalies critiques et solutions documentées (7 incidents)
- ✓ P6.1 — Index de la documentation (12 livrables)

✓ P6.2 — Monitoring Prometheus/Grafana décrit + screenshot dashboard

✓ P6.3 — Bilan technique critique (points forts + axes d'amélioration)

Vérification du dépôt GitHub

✓ Pipeline CI/CD vert sur branche main

✓ 29 tests PHPUnit passants en CI

✓ Fichier `.env.example` présent — `.env` dans `.gitignore`

✓ Schéma SQL commenté (`sql/schema.sql`)

✓ Images ERD, architecture, Grafana dans `docs/images/`

✓ Application accessible en production (<https://omar05.alwaysdata.net/cyna/>)

✓ Aucune clé secrète ou credential dans l'historique git
